Arquitectura del Sistema Farmacia

Versión: 0.1.0
Fecha: Abril 2026
Autor: Tobias Choclin

---

1. Introducción

Sistema de gestión farmacéutica modular que ofrece dos interfaces distintas (desktop y web) compartiendo la misma lógica de negocio. Diseñado para trazabilidad de inventario por lotes con control automático de vencimientos y estrategia FEFO (First Expired, First Out).

Objetivos del Sistema

- **Gestión integral** de inventario farmacéutico con trazabilidad por lotes
- **Control automatizado** de vencimientos y stock mínimo
- **Multi-plataforma**: Aplicación local (JavaFX) + Demo web 24/7 (Spring Boot)
- **Auditoría completa** de todos los movimientos de stock

---

2. Estructura del Proyecto

2.1 Arquitectura Multi-Módulo Maven

```
farmacia-parent/
│
├── pom.xml                    # POM padre con versiones centralizadas
│
├── farmacia-core/             # Módulo de lógica compartida
│   ├── src/main/java/
│   │   └── com/tobias/
│   │       ├── dao/          # Acceso a datos (DAOs)
│   │       ├── service/      # Lógica de negocio
│   │       ├── model/        # Entidades del dominio
│   │       ├── db/           # Gestión de conexiones
│   │       └── util/         # Utilidades (Excel, DatabaseDialect)
│   └── src/main/resources/
│       └── db/migration/
│           ├── sqlite/       # Migraciones para SQLite
│           └── postgresql/   # Migraciones para PostgreSQL
│
├── farmacia-desktop/          # Módulo JavaFX (aplicación local)
│   ├── src/main/java/
│   │   └── com/tobias/ui/
│   │       ├── controller/   # Controllers JavaFX
│   │       └── view/         # Lógica de vistas
│   └── src/main/resources/
│       ├── fxml/             # Archivos FXML (UI)
```

```
│       └── css/              # Estilos JavaFX
│
└── farmacia-web/              # Módulo Spring Boot (web)
    ├── src/main/java/
    │   └── com/tobias/web/
    │       └── controller/   # REST Controllers
    └── src/main/resources/
        ├── templates/        # Vistas Thymeleaf
        ├── static/css/       # CSS web
        └── application.properties
```

2.2 Separación de Responsabilidades

Módulo	Responsabilidad	Dependencias UI	Base de Datos
farmacia-core	Lógica de negocio, DAOs, servicios, modelos	❌ Ninguna	Agnóstica (SQLite/PostgreSQL)
farmacia-desktop	Interfaz JavaFX, controllers UI desktop	✅ JavaFX 22	SQLite local
farmacia-web	REST API, vistas Thymeleaf, controllers web	✅ Spring Boot	PostgreSQL (prod), SQLite (dev)

3. Capa de Datos

3.1 Modelo de Datos

```
┌──────────────────┐
│   productos      │
├──────────────────┤
│ id (PK)          │
│ codigo_barra     │
│ nombre           │
│ unidad_base      │
│ unidades_x_caja  │
│ stock_minimo     │
│ activo           │
└──────────────────┘
         │
         │ 1:N
         ↓
┌──────────────────┐
│     lotes        │
├──────────────────┤
│ id (PK)          │
│ producto_id(FK)  │
│ fecha_lote       │
│ fecha_vencim.    │
└──────────────────┘
         │
```

```
            │ 1:1
            ↓

┌──────────────────┐          ┌──────────────────┐
│  stock_lote      │          │ movimientos_stock│
├──────────────────┤          ├──────────────────┤
│ lote_id (PK,FK)  │          │ id (PK)          │
│ cantidad_base    │          │ tipo             │
└──────────────────┘          │ usuario          │
                              │ producto_id (FK) │
                              │ lote_id (FK)     │
                              │ cantidad_base    │
                              │ motivo           │
                              │ fecha_hora       │
                              └──────────────────┘
```

3.2 Estrategia de Base de Datos

farmacia-desktop → SQLite

- **Ubicación**: `~/.farmacia/data/farmacia.db`
- **Razón**: Base de datos embebida, sin servidor externo
- **Migraciones**: `db/migration/sqlite/`
- **Características SQLite**:
  - `INTEGER PRIMARY KEY AUTOINCREMENT`
  - `date('now')`, `datetime('now')`
  - Booleans como INTEGER (0/1)

farmacia-web → PostgreSQL (Producción)

- **Ubicación**: Servidor remoto (Koyeb, Neon, Supabase, etc.)
- **Razón**: Base de datos robusta, escalable, sin cold starts
- **Migraciones**: `db/migration/postgresql/`
- **Características PostgreSQL**:
  - `SERIAL PRIMARY KEY`
  - `CURRENT_DATE`, `CURRENT_TIMESTAMP`
  - `INTERVAL '30 days'`
  - BOOLEAN nativo

farmacia-web → SQLite (Desarrollo Local)

- **Activación**: Perfil `dev` (`-Dspring-boot.run.profiles=dev`)
- **Ubicación**: `~/.farmacia/data/farmacia.db`
- **Razón**: Testing local sin necesidad de PostgreSQL

3.3 Abstracción de Dialectos SQL

Clase `DatabaseDialect.java` para compatibilidad entre SQLite y PostgreSQL:

```
// Detecta automáticamente el tipo de BD
DatabaseType type = DatabaseDialect.detect(connection);

// Genera SQL específico del dialecto
String currentDate = DatabaseDialect.currentDate(connection);
// SQLite: "date('now')"
// PostgreSQL: "CURRENT_DATE"
```

---

4. Capa de Lógica de Negocio

4.1 Servicios Principales

StockService

Responsabilidad: Gestión de entradas y salidas de inventario

Métodos clave:

- `registrarEntrada(usuario, items)` → Crea/actualiza lotes, suma stock
- `previewSalidaFefo(productoId, cantidad)` → Simula salida con FEFO
- `registrarSalidaFefo(usuario, productoId, cantidad, motivo)` → Ejecuta salida FEFO
- `registrarSalidaPorLotes(usuario, motivo, asignaciones)` → Salida manual por lotes

Algoritmo FEFO:

```
-- Ordena lotes por vencimiento (más próximo primero)
ORDER BY
  fecha_vencimiento IS NULL,  -- NULL al final
  fecha_vencimiento,          -- Ordenar por fecha
  fecha_lote,                 -- Desempate por lote
  id                          -- Desempate por ID
```

ProductDao

Responsabilidad: CRUD de productos

Métodos clave:

- `findAll(filtro)` → Lista productos (con búsqueda opcional)
- `findByCodigoBarra(codigo)` → Busca por código de barras
- `insert(product)`, `update(product)`, `delete(id)` → ABM

4.2 Flujos de Datos

Flujo de Entrada de Mercancía

```
Usuario ingresa datos
     ↓
StockService.registrarEntrada()
     ↓
[Transacción BEGIN]
     ↓
For cada item:
  1. ensureLote() → Busca o crea lote
  2. addStock() → UPSERT en stock_lote (suma cantidad)
  3. insertMovimiento() → Registra auditoría
     ↓
[COMMIT]
```

Flujo de Salida FEFO

```
Usuario solicita salida
     ↓
previewSalidaFefo(productoId, cantidad)
     ↓
SELECT lotes con stock > 0
ORDER BY fecha_vencimiento ASC
     ↓
Asigna cantidad a cada lote (FEFO)
     ↓
¿Stock suficiente?
  NO → Exception
  SÍ → registrarSalidaPorLotes()
     ↓
[Transacción BEGIN]
For cada asignación:
  1. UPDATE stock_lote SET cantidad = cantidad - X
  2. insertMovimiento() → Auditoría
     ↓
[COMMIT]
```

---

5. Capa de Presentación

5.1 farmacia-desktop (JavaFX)

Arquitectura MVC

```
FXML (View)
   ↓ fx:controller
Controller
   ↓ usa
Service/DAO
```

```
    ↓ accede
Database (SQLite)
```

Vistas Principales

Vista	Archivo FXML	Controller	Función
Dashboard	`dashboard.fxml`	`DashboardController`	KPIs, alertas, gráficos
Productos	`productos.fxml`	`ProductController`	ABM productos
Stock	`stock.fxml`	`StockController`	Consulta stock actual
Vencimientos	`vencimientos.fxml`	`ExpiryController`	Alertas de vencimiento
Entradas	`entrada.fxml`	`EntradaController`	Registro de entradas
Salidas	`salida.fxml`	`SalidaController`	Salidas FEFO/manuales
Movimientos	`movimientos.fxml`	`MovimientosController`	Historial auditoría

Características UI

- **Búsqueda en tiempo real** con TableView
- **Validación inline** de formularios
- **Exportación Excel** de inventarios y reportes
- **Gráficos** de tendencias (próximamente)

5.2 farmacia-web (Spring Boot + Thymeleaf)

Arquitectura REST

```
HTTP Request
    ↓
@RestController / @Controller
    ↓ usa
Service/DAO
    ↓ accede
Database (PostgreSQL)
    ↓ retorna
JSON (API) o Model (Thymeleaf)
    ↓
Response (JSON o HTML)
```

Endpoints

Vistas HTML (Thymeleaf):

- `GET /` → Dashboard
- `GET /productos` → Gestión productos

- `GET /stock` → Consulta stock
- `GET /vencimientos` → Alertas vencimientos

API REST (JSON):

- `GET /api/dashboard/kpis` → KPIs principales
- `GET /api/dashboard/proximos-vencer` → Top 10 próximos vencer
- `GET /api/dashboard/stock-bajo` → Top 10 stock bajo
- `GET /api/productos` → Lista productos
- `GET /api/productos/codigo/{codigo}` → Producto por código
- `POST /api/productos` → Crear producto
- `PUT /api/productos/{id}` → Actualizar producto
- `DELETE /api/productos/{id}` → Eliminar producto
- `GET /api/stock?filtro={texto}` → Stock con filtro
- `GET /api/stock/lotes?productoId={id}` → Lotes de producto
- `GET /api/vencimientos?dias={90}` → Productos que vencen en X días
- `GET /api/vencimientos/stats` → Estadísticas vencimientos

Estética Idéntica

- **Paleta de colores**:
  - Primary: `#5B7FFF` (azul)
  - Success: `#28A745` (verde)
  - Warning: `#FFC107` (amarillo)
  - Danger: `#DC3545` (rojo)
  - Background: `#F8F9FA`
- **CSS convertido** desde JavaFX a web manteniendo exactamente los mismos estilos

---

6. Deployment

6.1 farmacia-desktop - Aplicación Local

Build

```
mvn clean package -pl farmacia-desktop -am
```

Empaquetado

- **Plugin**: maven-shade-plugin
- **Salida**: JAR "fat" con todas las dependencias incluidas
- **Tamaño**: ~15 MB (incluye JavaFX, SQLite, POI, etc.)

Ejecución

```
java -jar farmacia-desktop/target/farmacia-desktop-0.1.0.jar
```

Distribución

- JAR ejecutable standalone
- No requiere instalación
- Base de datos se crea automáticamente en `~/.farmacia/data/`

6.2 farmacia-web - Demo Web 24/7

Build

```
mvn clean package -pl farmacia-web -am
```

Dockerfile (Multi-stage)

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk-jammy AS build
WORKDIR /build
COPY pom.xml .
COPY farmacia-core/ farmacia-core/
COPY farmacia-web/ farmacia-web/
RUN mvn clean package -pl farmacia-web -am -DskipTests

# Stage 2: Runtime
FROM eclipse-temurin:21-jre-jammy
RUN useradd -m -u 1000 farmacia
USER farmacia
WORKDIR /app
COPY --from=build /build/farmacia-web/target/farmacia-web-*.jar app.jar
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s \
  CMD curl -f http://localhost:8080/api/dashboard/kpis || exit 1
ENTRYPOINT exec java $JAVA_OPTS -jar app.jar
```

Optimizaciones:

- Multi-stage build (reduce tamaño final ~70%)
- Non-root user (seguridad)
- Health check integrado
- Variables JAVA_OPTS para tuning

Despliegue en Koyeb

Requisitos:

- Imagen Docker (Docker Hub, GitHub Container Registry)

- Base de datos PostgreSQL (Neon, Supabase, Koyeb managed)

Variables de entorno:

```
DATABASE_URL=jdbc:postgresql://host:5432/farmacia
DATABASE_USER=postgres
DATABASE_PASSWORD=secret
JAVA_OPTS=-Xmx512m -Xms256m
```

Características Koyeb:

- ✅ Free tier generoso
- ✅ Sin cold starts (vs Render)
- ✅ Health checks automáticos
- ✅ Auto-deploy desde GitHub
- ✅ SSL/HTTPS incluido

Docker Compose (Testing Local)

```
docker-compose up --build
# Levanta PostgreSQL + farmacia-web
# Accesible en http://localhost:8080
```

---

7. Tecnologías y Dependencias

7.1 Stack Tecnológico

Categoría	Tecnología	Versión	Uso
Lenguaje	Java	21 LTS	Lenguaje principal
Build	Maven	3.9+	Gestión dependencias
Desktop UI	JavaFX	22.0.1	Interfaz gráfica nativa
Web Framework	Spring Boot	3.2.4	Backend web
Web Templating	Thymeleaf	-	Renderizado HTML
BD Embedded	SQLite	3.46.0.0	Desktop + dev local
BD Relacional	PostgreSQL	16+	Producción web
Migraciones	Flyway	10.15.0	Versionado schema
Exportación	Apache POI	5.2.5	Generación Excel
Logging	SLF4J + Logback	2.0.13	Logs aplicación

7.2 Dependencias por Módulo

farmacia-core

```
<!-- Base de datos -->
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>

<!-- Utilidades -->
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
</dependency>
```

farmacia-desktop

```
<!-- UI -->
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
</dependency>

<!-- Core -->
<dependency>
  <groupId>com.tobias</groupId>
  <artifactId>farmacia-core</artifactId>
</dependency>
```

farmacia-web

```
<!-- Spring Boot -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<!-- Base de datos -->
```

```xml
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>

<!-- Core (con exclusión de slf4j-simple) -->
<dependency>
  <groupId>com.tobias</groupId>
  <artifactId>farmacia-core</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

---

8. Características Funcionales

8.1 Gestión de Inventario

Entrada de Mercancía

- Registro por lotes con fecha de fabricación y vencimiento
- Asociación automática a productos existentes
- Creación de lotes nuevos si no existen
- Incremento automático de stock
- Auditoría completa de movimientos

Salida de Mercancía

Modo FEFO (Automático):

- Selección automática de lotes más próximos a vencer
- Preview de asignación antes de confirmar
- Validación de stock suficiente
- Decremento automático por lote

Modo Manual:

- Selección manual de lotes específicos
- Útil para devoluciones o ajustes
- Validación individual por lote

8.2 Control de Vencimientos

Alertas Automáticas

- **Vencidos**: Productos con fecha < hoy (rojo)

- **Vence muy pronto**: ≤ 7 días (rojo)
- **Vence pronto**: 8-30 días (amarillo)
- **Vence futuro**: > 30 días (azul)

Dashboard de Vencimientos

- Estadísticas: vencidos, 7 días, 30 días, 90 días
- Listado detallado con días restantes
- Filtro por rango de días personalizables
- Exportación a Excel

8.3 Control de Stock

Stock Mínimo

- Definición de stock mínimo por producto
- Alertas automáticas cuando stock < mínimo
- Dashboard de productos con stock bajo
- Porcentaje de stock disponible vs mínimo

Estados de Stock

- **En Stock**: stock ≥ mínimo (verde)
- **Stock Bajo**: 0 < stock < mínimo (amarillo)
- **Sin Stock**: stock = 0 (rojo)

8.4 Auditoría

Registro de Movimientos

Todos los movimientos quedan registrados con:

- Tipo (ENTRADA, SALIDA)
- Usuario responsable
- Producto y lote afectados
- Cantidad
- Motivo
- Fecha y hora exacta

Trazabilidad Completa

- Historial de movimientos por producto
- Filtrado por fecha, tipo, usuario
- Exportación de reportes de auditoría

---

9. Seguridad y Buenas Prácticas

9.1 Seguridad

Base de Datos

- **Prepared Statements**: Prevención de SQL injection
- **Validación de entrada**: Sanitización de datos del usuario
- **Transacciones**: ACID compliance para consistencia

Web

- **Variables de entorno**: Credenciales nunca en código
- **Non-root user**: Container ejecuta como usuario no-privilegiado
- **Health checks**: Monitoreo automático de disponibilidad
- **HTTPS**: SSL/TLS en Koyeb (automático)

9.2 Manejo de Errores

Desktop

- Try-catch en todos los DAOs y servicios
- Mensajes de error amigables al usuario
- Logging de excepciones para debugging

Web

- Exception handlers globales en Spring
- Códigos HTTP apropiados (404, 400, 500)
- JSON de error estructurado para API

9.3 Testing

Estrategia

- Tests unitarios para lógica de negocio
- Tests de integración para DAOs
- Testing manual de UI

Ambiente de Desarrollo

- Perfil `dev` con SQLite para testing rápido
- Docker Compose para testing con PostgreSQL
- Datos de prueba reproducibles

---

10. Escalabilidad y Futuro

10.1 Extensibilidad

Nuevos Módulos

La arquitectura permite agregar fácilmente:

- `farmacia-mobile` (React Native, Flutter)
- `farmacia-api` (API standalone sin UI)
- `farmacia-analytics` (Módulo de reportes avanzados)

Todos reutilizando `farmacia-core`

Nuevas Funcionalidades

- **Ventas**: Registro de ventas a clientes
- **Proveedores**: Gestión de proveedores y órdenes de compra
- **Recetas**: Control de medicamentos con receta
- **Usuarios**: Sistema de roles y permisos
- **Reportes**: Gráficos avanzados, exportación PDF
- **Notificaciones**: Email/SMS para alertas

10.2 Migración de Datos

SQLite → PostgreSQL

```
# Exportar desde SQLite
sqlite3 farmacia.db .dump > dump.sql

# Adaptar sintaxis (herramientas o manual)
# - AUTOINCREMENT → SERIAL
# - INTEGER(0/1) → BOOLEAN

# Importar a PostgreSQL
psql -U postgres -d farmacia < dump_adapted.sql
```

10.3 Performance

Optimizaciones Aplicadas

- **Índices** en columnas de búsqueda frecuente
- **COALESCE** para evitar NULL checks en queries
- **Transacciones** para operaciones batch
- **Connection pooling** (HikariCP en Spring Boot)

Métricas Objetivo

- **Desktop**: Respuesta instantánea (<100ms)
- **Web**: API response < 500ms, páginas < 2s
- **Database**: Queries < 100ms (índices apropiados)

---

11. Conclusión

Fortalezas del Diseño

✅ **Modularidad**: Separación clara de responsabilidades
✅ **Reutilización**: farmacia-core compartido entre módulos
✅ **Flexibilidad**: Soporta SQLite y PostgreSQL transparentemente
✅ **Escalabilidad**: Arquitectura preparada para crecimiento
✅ **Portabilidad**: Desktop (local) + Web (cloud) con misma lógica
✅ **Mantenibilidad**: Código organizado, patrones consistentes
✅ **Auditabilidad**: Trazabilidad completa de movimientos

Decisiones Arquitectónicas Clave

1. **Multi-módulo Maven**: Evita duplicación, facilita evolución
2. **Database abstraction**: DatabaseDialect permite cambio de motor sin refactorizar lógica
3. **Flyway migrations**: Versionado de schema, deploy confiable
4. **FEFO automático**: Minimiza pérdidas por vencimiento
5. **Dual deployment**: Desktop para usuarios locales, Web para portfolio/demo

---

Apéndice A: Comandos Útiles

Build y Ejecución

```
# Compilar todo el proyecto
mvn clean package

# Ejecutar farmacia-desktop
mvn javafx:run -pl farmacia-desktop
# O con el JAR
java -jar farmacia-desktop/target/farmacia-desktop-0.1.0.jar

# Ejecutar farmacia-web (desarrollo con SQLite)
mvn spring-boot:run -pl farmacia-web -Dspring-boot.run.profiles=dev

# Ejecutar farmacia-web (producción con PostgreSQL)
mvn spring-boot:run -pl farmacia-web

# Build Docker
docker build -t farmacia-web:latest .

# Testing local con Docker Compose
docker-compose up --build
```

Base de Datos

```
# Conectar a SQLite
sqlite3 ~/.farmacia/data/farmacia.db

# Conectar a PostgreSQL (local)
psql -U postgres -d farmacia
```

```
# Ver migraciones aplicadas
SELECT * FROM flyway_schema_history;
```

Apéndice B: Recursos

Documentación

- `README.md` - Introducción general
- `DEPLOYMENT.md` - Guía de despliegue completa
- `KOYEB_DEPLOY.md` - Despliegue específico en Koyeb
- `LOCAL_TESTING.md` - Testing local con Docker
- `.env.example` - Template de variables de entorno

Enlaces Útiles

- JavaFX: https://openjfx.io/
- Spring Boot: https://spring.io/projects/spring-boot
- Flyway: https://flywaydb.org/
- Koyeb: https://www.koyeb.com/
- PostgreSQL: https://www.postgresql.org/

Fin del Documento